

1. INDEXES

-- Speed up queries on movies titles

```
CREATE INDEX idx_movie_title ON MOVIES(MovieTitle);
```

-- Speed up queries screening schedules by day/time

```
CREATE INDEX idx_screening_datetime
```

```
ON SCREENINGS(ScreeningDate, ScreeningTime);
```

-- Speed up queries ticket by screening

```
CREATE INDEX idx_ticket_screening
```

```
ON TICKETS(ScreeningID);
```

TEST INDEX

```
SELECT * FROM MOVIES
```

```
WHERE MovieTitle = 'Demon Slayer: Mugen Train';
```

	MovieID	MovieTitle	Genre	DurationMinutes
▶	10	Demon Slayer: Mugen Train	Animation	117
*	NULL	NULL	NULL	NULL

```
SELECT * FROM SCREENINGS
```

```
WHERE ScreeningDate = '2026-04-10';
```

	ScreeningID	ScreeningTime	ScreeningDate	MovieID	RoomID
▶	42	13:30	2026-04-10	9	6
	12	14:30	2026-04-10	10	7
*	NULL	NULL	NULL	NULL	NULL

```
SHOW INDEX FROM MOVIES;
```

```
EXPLAIN SELECT * FROM MOVIES WHERE MovieTitle = 'Oppenheimer';
```

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	MOVIES	NULL	ref	idx_movie_title	idx_movie_title	802	const	1	100.00	NULL

2. VIEWS

-- View provide summaries of daily screenings

CREATE VIEW DailyScreenings AS

SELECT

s.ScreeningID,
m.MovieTitle,
s.ScreeningDate,
s.ScreeningTime,
c.RoomName

FROM SCREENINGS s

JOIN MOVIES m ON s.MovieID = m.MovieID

JOIN CINEMAROOMS c ON s.RoomID = c.RoomID;

-- View the number of tickets sold

CREATE VIEW SeatCount AS

SELECT

ScreeningID,
COUNT(*) AS TicketsSold

FROM TICKETS

GROUP BY ScreeningID;

-- TEST VIEW

SELECT * FROM DailyScreenings LIMIT 10;

	ScreeningID	MovieTitle	ScreeningDate	ScreeningTime	RoomName
▶	3	Avengers: Endgame	2026-04-18	14:00	Room 1
	9	Godzilla Minus One	2026-04-23	21:30	Room 1
	39	Dune: Part Two	2026-04-23	18:45	Room 1
	44	Suzume	2026-04-18	17:15	Room 1
	19	Demon Slayer: Mugen Train	2026-04-26	21:15	Room 2
	21	Demon Slayer: Mugen Train	2026-04-11	17:45	Room 2
	26	Oppenheimer	2026-04-17	11:00	Room 2
	40	Suzume	2026-04-28	15:30	Room 2
	47	Oppenheimer	2026-04-09	20:30	Room 2
	2	Oppenheimer	2026-04-25	10:45	Room 3

SELECT * FROM SeatCount;

	ScreeningID	TicketsSold
▶	1	14
	2	13
	3	17
	4	6
	5	11
	6	6
	7	12
	8	13
	9	7
	10	5

3. STORED PROCEDURE

DELIMITER //

CREATE PROCEDURE BookTicket(

IN p_customer INT,

IN p_screening INT,

IN p_seat VARCHAR(10)

)

BEGIN

INSERT INTO TICKETS (SeatNumber, ScreeningID, CustomerID)

VALUES (p_seat, p_screening, p_customer);

SELECT 'Booking successful!' AS Message;

```
END //
```

```
DELIMITER ;
```

-- TEST PROCEDURE

```
CALL BookTicket(1, 1, 'A10');
```

```
SELECT * FROM TICKETS
```

```
ORDER BY TicketID DESC
```

```
LIMIT 5;
```

	TicketID	SeatNumber	ScreeningID	CustomerID
▶	511	A10	1	1
	510	F1	50	102
	509	A2	26	255
	508	J10	1	423
	507	L3	48	393
*	NULL	NULL	NULL	NULL

4. FUNCTION

```
DELIMITER //
```

```
CREATE FUNCTION GetOccupancyRate(p_screening INT)
```

```
RETURNS FLOAT
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    DECLARE totalSeats INT;
```

```
    DECLARE soldSeats INT;
```

```
    SELECT Capacity INTO totalSeats
```

```
    FROM CINEMAROOMS cr
```

```
    JOIN SCREENINGS s ON cr.RoomID = s.RoomID
```

```
WHERE s.ScreeningID = p_screening;
```

```
SELECT COUNT(*) INTO soldSeats
```

```
FROM TICKETS
```

```
WHERE ScreeningID = p_screening;
```

```
RETURN (soldSeats / totalSeats) * 100;
```

```
END //
```

```
DELIMITER ;
```

```
-- TEST FUNCTION
```

```
SELECT GetOccupancyRate(1) AS OccupancyRate;
```

	OccupancyRate
▶	18.75

```
SELECT
```

```
ScreeningID,
```

```
GetOccupancyRate(ScreeningID) AS Rate
```

```
FROM SCREENINGS
```

```
LIMIT 5;
```

	ScreeningID	Rate
▶	3	14.1667
	15	13.3333
	32	16.6667
	45	3
	2	13

5. TRIGGER

```
DELIMITER //
```

```
CREATE TRIGGER PreventDuplicateSeat
```

```
BEFORE INSERT ON TICKETS
```

```

FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1 FROM TICKETS
        WHERE ScreeningID = NEW.ScreeningID
        AND SeatNumber = NEW.SeatNumber
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Seat already booked!';
    END IF;
END //

```

DELIMITER ;

-- TEST TRIGGER

```

INSERT INTO TICKETS (SeatNumber, ScreeningID, CustomerID)
VALUES ('B1', 1, 1);

```

```

INSERT INTO TICKETS (SeatNumber, ScreeningID, CustomerID)
VALUES ('B1', 1, 2);

```

✖ 76 00:02:26 INSERT INTO TICKETS (SeatNumber, ScreeningID, CustomerID) VALU... Error Code: 1644. Seat already booked! 0.000 sec

6. DEMO QUERY

```

SELECT
    c.CustomerName,
    m.MovieTitle,
    s.ScreeningDate,
    s.ScreeningTime,
    t.SeatNumber
FROM TICKETS t

```

JOIN CUSTOMERS c ON t.CustomerID = c.CustomerID

JOIN SCREENINGS s ON t.ScreeningID = s.ScreeningID

JOIN MOVIES m ON s.MovieID = m.MovieID

LIMIT 10;

	CustomerName	MovieTitle	ScreeningDate	ScreeningTime	SeatNumber
►	Sarah Moore	Avengers: Endgame	2026-04-18	14:00	E3
	Alice Martinez	Avengers: Endgame	2026-04-18	14:00	E6
	Sarah Smith	Avengers: Endgame	2026-04-18	14:00	B5
	Lucas Anderson	Avengers: Endgame	2026-04-18	14:00	D2
	Leo Wilson	Avengers: Endgame	2026-04-18	14:00	C3
	Sophia Martinez	Avengers: Endgame	2026-04-18	14:00	E1
	Liam Davis	Avengers: Endgame	2026-04-18	14:00	A3
	Oliver Taylor	Avengers: Endgame	2026-04-18	14:00	C2
	Daniel Thomas	Avengers: Endgame	2026-04-18	14:00	C2
	Noah Moore	Avengers: Endgame	2026-04-18	14:00	B1